Scripting Languages

What is a scripting language?

No precise definition.

Generally, two ancestors for this paradigm:

- Shell languages (`sh`, `csh`, `bash`)
- Tools for text processing and report generation (`sed` and `awk`)

Common characteristics

Both batch and interactive use

Economy of expression

Lack of declarations; simple scoping rules.

Flexible dynamic typing

Easy access to other programs

Sophisticated pattern matching and string manipulation

High level data types

Not so easily defined

Some general purpose languages (like scheme or Visual Basic) are used for scripting purposes.

Some scripting languages (like Python, Ruby, Perl and Javascript) are intended to be used as general purpose languages.

Scripting languages are often thought in contrast to systems programming languages.

Javascript

The good, the bad, the ugly

Why Javascript?

https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language

Javascript - history

- Created by Brendan Eich in 1995
- Eich was originally hired by Netscape to add the Scheme programming language to websites in order to make them dynamic

- Design goal was to match the ‘functions’ of Scheme, the Object Orientation of ‘Self’ and the syntax of Java.

- Original version of Javascript was designed, implemented and shipped in 10 days!

Scheme + Self + Java + 10 days of design and implementation = ???

- Really awesome:
  - Functions!
  - Huge community
- Pretty good:
  - Prototypal inheritance
  - Objects and JSON
  - Regular expressions
- Not good:
  - ==, eval, new, void
- Even worse:
  - Global variables, scope, semicolon insertion, NaN, coersions

First…

Javascript is intimidating due to its vast syntax

It is also widely considered a beginner language

Javascript has been added to and added to, but rarely subtracted from.

Despite this, “modern” JS often features complex syntax mixed with complex design patterns. Programmers also have their “preferences”

There are many differing opinions about what the ‘right’ JS looks like.

As with any PL, its most important to be consistent within a single codebase.

First…

Despite being considered a entry level PL, there are many important aspects of the semantics of JS.

It is closer to a functional language than a imperative language.

Consider:

```
const sum = (...nums) => {
    return nums.reduce((a, b) => a + b, 0);
};
```

Contains 2 anonymous functions, higher order functions, spread, arrow syntax…

First…

Key to understand javascript is the terminology used to describe its evolution over time.

All programming languages do this. However, JS has evolved significantly compared to other examples.

Java 4 added Generics. Java 8 added lambdas. Java 17 added enhanced `switch`

C has K&R C, ANSI C, ISO C, C98, C11, C23

Javascript has ES5, ES6, ES16 etc. ES is short for ECMAScript.

Language Features

Modules

Modules provide a simple way of organizing multiple files that contain source code

Modules in JS were actually implementation specific until the release of ES6

- (CommonJS, and now ECMAScript standard imports)
- CommonJS
- Ecma modules

Object Orientation

Javascript uses a new to you prototypal object system.

- Every object has a link to another object called a prototype (`__proto__`)
- That prototype is linked to its prototype (and so on until null)

Types are also dynamic, not static. The type of a type can change at runtime.

This is confusing for those new to the prototypal model.

Javascript decided to provide a lot of syntax to help hide how it actually is working.

Objects

Objects can be created used in 3 main ways:

- Prototypal
- Functional
- ES6+ Class Syntax

Object Oriented Programming

Each of these methods allow subtype polymorphism (inheritance)

- Pseudoclassical approach
- Functional approach
- Class approach

But wait, there's more!

- Compositional inheritance (not covered, similar to Go struct embedding)

Encapsulation

Not all object creation methods in JS allow private members.

- Strictly adhering to the prototypal model has no object privacy
- Privacy can be achieved using a functional closure
  - See functional-class.js
- Newer than ES22 classes allow private methods and fields (with strange syntax: names begin with #)
- This is essentially the same effect as the functional closure

Functional Programming

Functional Programming

Functions are first class objects, and can be passed and returned from functions

- sort.js

Functional scope was the initial design intention

- functional-class.js and why.js

Closures

How does this work?

Asynchronous coding with functions

V8 interpreter does not run code in multiple threads (well, sort of)!

Instead, the event loop allows non-blocking i/o operations to happen.

Two main ways to handle non-blocking I/O

- Callback functions
- async / await with promises

Example:

- Reading from a file

Modern Javascript

Strict mode

Many new features of the language are built on top of “strict” mode, added in ES5

From Mozilla’s page:

a. Eliminates some JavaScript silent errors by changing them to throw errors.
b. Fixes mistakes that make it difficult for JavaScript engines to perform optimizations: strict mode code can sometimes be made to run faster than identical code that's not strict mode.
c. Prohibits some syntax likely to be defined in future versions of ECMAScript.

Example

ECMAscript

ECMAscript is the adopted language standard required by implementers of Javascript, including NodeJS and your favorite web browser.

ES6 (ECMAScript 2015) introduced many needed features into the language

- Const, let, arrow functions, many more.

Since then, there have been yearly releases with new features added to the language. See https://en.wikipedia.org/wiki/ECMAScript_version_history

The next release is always referred to as ESNext

Linting

Another word for Static Code Analysis is called “linting”

Linting tools have been around for almost as long as programming languages

Linting is especially important for Javascript because of the huge variety of ways to program, and the huge amount of ways to make mistakes.

ESLint

ESLint is a static analyser for javascript, that is also highly configurable.

Any significant javascript project should be using ESLint as a requirement of its development.

P3 will require you to use ESLint.

See also https://github.com/airbnb/javascript

Example: eslint

Typescript

To get around many of the problems demonstrated in this presentation, a new language has become popular: Typescript.

Typescript is a statically-typed version of javascript, that transpiles to javascript.

The typescript compiler is written in Typescript, and can be compiled with itself for a javascript version. This can then be run in any javascript environment

Learning more

Highly recommended but out of date: Javascript, The Good Parts, by Douglas Crockford

Mozilla reference

Also, try it! Use one of the (many) popular frameworks to build something.

You can also use a NodeJS server as the host for your Demo Hour Presentations.

Historical Examples

Awesome parts

Functions!

Community

- Supported by all browsers
- NodeJS is one of the most popular runtimes in software development
- NPM hosts 1.3 million different packages (libraries)
- Tremendously fast paced framework evolution
    - NextJS, ReactJS, VueJS, AngularJS, JQuery, YUI (yahoo!), and many more

Pretty good

Scripting goodies like Regular expressions are built in

- regular-expressions.js

Objects and JSON

- JSON revolutionized web object formats (have you seen XML?)
- data.json example

OOP and Inheritance examples will come later…

Not good

Actually, a lot of things:

- `==` operator
- `void` is an operator!
- Hoisting
- Statements don’t require blocks
- Reserved words that do nothing
- And more…

See why.js to try for yourself!

Even worse

Out of everything, Javascript has some really terrible details.

- Global variables and common package structure
  - Led to the development of confusion CommonJS packages
- NaN
- Lack of block scope, despite having blocks (without `let` and `const`)
- Automatic semicolon insertion (language spec)